Australian National University

Venue _____

School of Computing

Student Number |__|__|__|__|__|__|__|__|

EXAMINATION

Semester 2 – Week 1, 2024

COMP2300_COMP6300_ENGN2219 (DA/PX)

Computer Organization and Program Execution

This paper is for ANU students.

Examination Duration: 180 minutes

Reading Time: 15 minutes

Exam Conditions:

No materials permitted

Materials Permitted In The Exam Venue:

(No electronic aids are permitted e.g. laptops, phones)

No materials permitted

Materials To Be Supplied To Students:

1 x 20 pages

Instructions To Students:

The exam is marked out of 100. Each question's total marks and the breakdown of marks are indicated before each question. You should spend more time on questions with more weight.

Q1	**Multiple Choice**	20	
Q2	**Digital Logic**	10	
Q3	**Timing Issues**	10	
Q4	**Finite State Machine**	14	
Q5	**ARM Assembly**	10	
Q6	**Input/Output**	6	
Q7	**Microarchitecture**	30	
Aggregate Marks		100	

- Write your student number and answers clearly in good handwriting.

- **An ARM cheat sheet is provided at the end of the exam sheet (after page # 21).**

- Any formula sheet is neither deemed necessary nor provided for this closed-book exam.

- All the arithmetic required to answer questions is simple as long as you are comfortable with the binary number system. The use of a calculator is not deemed necessary for this exam. Using a calculator or a mobile aid is strictly forbidden.

- This exam has seven main questions and several sub-questions. The first question consists of 20 multiple-choice and true/false questions on a variety of topics. The subject of each remaining question is indicated.

- Indicate your answers clearly, and do not make multiple selections. Think carefully before selecting your answer.

- Do not attach extra sheets. Write your answers in the space provided in the question sheet.

- We provide an extra sheet after the final exam question. If you use that extra sheet to provide answers to any questions, clearly mention page number 20 so we can mark your answers.

- In the upcoming question, there is only one correct answer. Circle only one answer.

- Clearly read the marking scheme of the first question. There is negative marking in the first question only. Avoid random guesses as it might negatively affect your score.

Multiple choice and true/false questions [20, 1 for correct, -1 for wrong, 0 for no selection]

Q1a. How many bits are needed to represent –512 with sign/magnitude numbers?

- 7
- 8
- 9
- 10
- None of the above

Q1b. If one of the inputs of an XNOR gate is connected to logic level 1 (TRUE) forever, the resulting circuit behaves like:

- OR gate
- NOR gate
- NOT gate
- Short circuit wire
- None of the above

Q1c. Under De Morgan's Theorem, which of the following circuits are equivalent:

- An NOR gate with inverted output *is equivalent to* an AND gate with one inverted input
- An AND gate with inverted output *is equivalent to* a NOR gate with inverted output
- An AND gate with inverted output *is equivalent to* an OR gate with inverted inputs
- An OR gate with inverted output *is equivalent to* an OR gate with inverted inputs
- A NAND gate with inverted output *is equivalent to* an OR gate with inverted inputs
- None of the above

Q1d. A 2:4 decoder can be used to implement an XNOR gate by connecting which ones of its one-hot outputs to the inputs of an OR gate:

- 00 and 01
- 00 and 11
- 01 and 10
- 10 and 11
- 00 and 10

Q1e. Two NAND gates are cross-coupled to produce outputs X and Y. Their remaining terminals are connected to inputs A and B. When A = 0 and B = 0, which one of the following statements is correct?

- X is 1, and Y is 0
- X is 0, and Y is 1
- X and Y remember their old values
- X and Y oscillate between 1 and 0
- X and Y are not complements of each other

Q1f. Whether the ALU uses ripple carry adder or carry save adder is determined by:

- ISA
- Microarchitecture
- None of the above

Q1g. Instruction width is determined by:

- ISA
- Microarchitecture
- None of the above

Q1h. Whether or not the CPU uses branch prediction is determined by:

- ISA
- Microarchitecture
- None of the above

Q1i. Whether or not control unit is hardwired or microprogrammed is determined by:

- ISA
- Microarchitecture
- None of the above

Q1j. What is the execution time (in seconds) of a program that executes 2 billion instructions on a CPU with a clock frequency of 2 GHz and an average CPI of 4?

- 0.5
- 1
- 2
- 4
- 8

Q1k. Energy consumption is determined by:

- Execution time only
- Power only
- Both the execution time and power
- None of the above

Q1l. A C program is compiled into ARM (A) and MIPS (M) assemblies. W and X are ARM CPUs with similar frequency. Y and Z are MIPS CPUs with similar frequency. The assembly program A is run on W and X, and M is run on Y and Z. In which of the following scenarios is it okay to use instructions per cycle or IPC as a metric for comparing the performance of two CPUs?

- A on W *versus* M on Y
- A on X *versus* M on Z
- A on W *versus* A on X
- A on X *versus* M on Y
- None of the above

Q1m. Which of the following metrics is NOT impacted by the algorithm?

- Energy
- Power
- Frequency
- Execution time
- None of the above

Q1n. Register R0 contains the index of an element in a C array. The base address of the array in memory is 0x0. Each array element is 16 bytes in size. Which of the following instructions writes the memory address of the array element in register `R1`?

- `LSL R1, R0, #2`
- `LSL R1, R0, #3`
- `LSL R1, R0, #4`
- `LSL R1, R0, #5`
- `LSL R1, R0, #6`

Q1o. Which of the following is an example of shifting the “engineering” burden from software to hardware?

- ISA with complex instructions
- ISA with simple instructions
- Dynamic scheduling
- Microprogrammed control unit
- Complex addressing modes

Q1p. Interrupt handers always execute when the CPU encounters the bx instruction:

- True
- False
- Need more information

Q1q. Which of the following instructions is an example of a false dependency?

- `ADD R0, R1, R2` *followed by* `ADD R3, R4, R5`
- `LDR R0, [R1]` *followed by* `STR R0, [R1]`
- `ADD R0, R1, R2` *followed by* `SUB R0, R4, #16`
- `ADD R0, R1, R2` *followed by* `ADD R1, R1, R7`
- None of the above

Q1r. A 2-bit branch predictor is more accurate and space-efficient than a 1-bit predictor:

- True
- False

Q1s. Consider a load (L) that incurs a cache miss, followed by an instruction (M) dependent on L, followed by an instruction (N) that does not depend on L or M. An in-order CPU will:

- Stall L and M and N
- Stall L, but not M and N
- Stall L and N, but not M
- Stall L and M, but not N
- None of the above

Q1t. In general, instruction-level parallelism is limited by:

- High frequency of independent add and subtract instructions
- High frequency of memory instructions that take 100s of cycles to resolve
- High frequency of young and independent ALU instructions
- High frequency of shift instructions

Digital Logic Fundamentals [2 + 4 + 2, 1 + 1]

Q2a. One day at work, you are tasked with designing a combinational circuit with a three-bit input, {A, B, C} (A is the most significant bit, and C is the least significant bit), and one output, Y. The value of Y is 1 when the 3-bit input contains two or more ones.

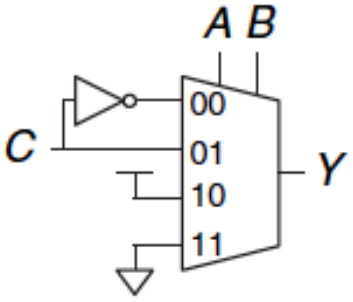
Answer the next three questions. You can also use the white space on the next page if required.

A. Draw the truth table for the combinational circuit.

B. Express the output Y as the simplest sum of product representation. Show your work step-by-step. Your work should first show the expanded Boolean equations translated from the truth table.

C. Using basic logic gates, draw a schematic of Y as a function of A, B, and C.

Q2b. The following circuit shows an example of implementing logic using multiplexers. The “10” input of the multiplexer is connected to logic level 1 or TRUE, and the input “11” is connected to logic level Ground or 0. Determine the output Y for the following two cases. Write your answer next to the options below. You can assume that A is the most significant bit.

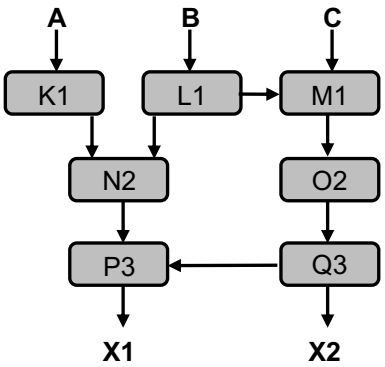


A. $A = 0$, $B = 1$, $C = 0$, $Y =$ ____

B. $A = 0$, $B = 0$, $C = 1$, $Y =$ ____

Timing in Digital Circuits [2 + 2 + 2, 4]

Q3a. Consider the high-level schematic of a general-purpose combinational hardware circuit shown that computes two outputs from three inputs. Circle the correct answer.



A. The correct statement about the relationship of X1 and X2 to C is:

   a. X1 and X2 both depend on C

   b. Only X1 depends on C

   c. Only X2 depends on C

   d. None of the above is correct

B. If all components have a delay of 1 unit, then the critical path of X1 consists of:

   a. K1, N2, P3

   b. L1, M1, O2, Q3, P3

   c. L1, N2, P3

   d. L1, N2, Q3, P3

   e. M1, O2, Q3, P3

C. If K1 has a delay of 4 units, and all other components have a delay of 1 units, then the critical path of X1 consists of:

   a. K1, N2, P3

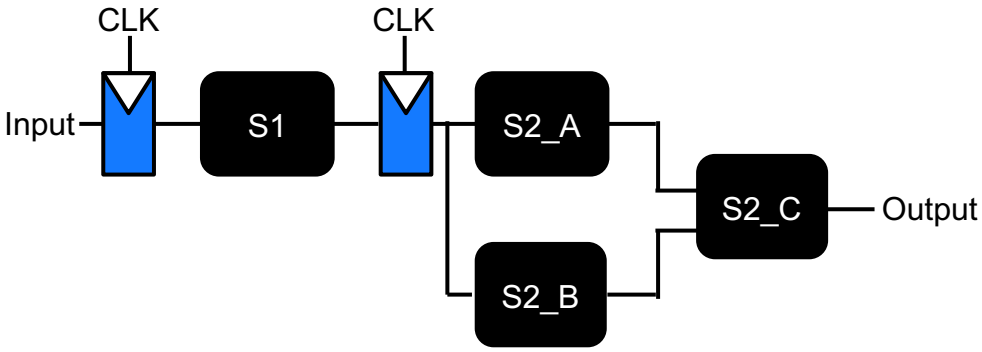
   b. L1, M1, O2, Q3, P3

   c. L1, N2, P3

   d. L1, N2, Q3, P3

   e. M1, O2, Q3, P3

Q3b. The following sequential circuit processes its input in a staged manner. The combinational elements are labeled S1, S2_A, S2_B, and S2_C. The registers receive the clock (CLK) input. The circuit shown is a purpose-built accelerator for solving a high-performance computational problem. Note that the second stage is divided into several combinational circuits labeled A, B, and C. Assume a new input is available to S1 at the rising edge of a clock cycle. The circuitry (not shown) receiving the output signal, Output, stores it in a register for one full clock cycle. The timing specification for the combinational and sequential elements of the circuit is given below. What is the time (in nanoseconds) that it takes to process 25 inputs?



Timing specification

- The propagation delay of S1 is equal to 4.4 nanoseconds.
- The propagation delay of Stage 2 (A) is equal to 3.2 nanoseconds.
- The propagation delay of Stage 2 (B) is equal to 4 nanoseconds.
- The propagation delay of Stage 2 (C) is equal to 0.3 nanoseconds.
- The clock-to-Q propagation delay (t_{pcq}) is equal to 0.5 nanoseconds.
- The setup time (t_{setup}) is equal to 0.2 nanoseconds.
- The hold time (t_{hold}) is zero.

Write your answer below.

Finite State Machines [5 + 3 + 4 + 2]

Q4. Design a finite state machine (FSM) with one input, A, and one output, X. The output X should be 1 if A has been 0 for two consecutive clock cycles. Otherwise, X should be 0.

A. Design the Moore state transition diagram for the above specification.

B. Build the encoded state transition tables.

C. Specify and simplify the next state and output equations.

D. Draw the high-level schematic of the FSM. You do not need to show the gate-level logic, but you should show the state register, next stage component, feedback loop, and output component.

Write your answers on this page and the next page.

ARM Assembly and ISA [10]

Q5. Translate the following C function (starting at the label `func`) into 32-bit ARM assembly code. The parameters of the function are provided in `R0` and `R1`. The result should be returned in `R0`.

Your code should preserve registers using the ARM calling convention, if necessary.

```
int func(int n) {
      int i = 0;
      int pow = 1;

      while (pow != n) {
            pow = pow * 2;
            i = i + 1;
     }
    return i;
```

Write your code in the space provided on this page and the next page (if needed).

Input/Output [3, 3]

Q6a. It has been observed that at a bank, during peak hours, the server processing client requests spends 50% of the time where the CPU is 100% busy and 50% of the time waiting for a response from disk. If the CPU is upgraded to make it 400% faster than the existing CPU, what total speedup can the banking service expect?

Q6b. An embedded computer system uses memory-mapped I/O devices. A specific I/O device is assigned the memory address 0x50000000. An I/O register is associated with the memory address, and its output connects to the input pins of the I/O device. If the 5th bit of the I/O register is set, the I/O device goes into standby mode. Show the 32-bit ARM assembly code for sending the I/O device into standby mode.

Microarchitecture [5 + 5 + 5, 3, 6 + 2 + 2 + 2]

Q7a. This question is about executing the following instruction sequence on a *scalar* in-order pipelined CPU. First, study the instruction sequence below. Then, answer the questions.

```
i1:   MOV R0, #10
i2:   MOV R1, #0
i3:   MOV R2, #256
i4:   MOV R3, #0
i5:   MOV R4, #0
     LOOP:
i6:   CMP R0, R1
i7:   BEQ DONE
i8:   LDR R3, [R2], 4
i9:   LDR R4, [R3, #0]
i10:  ADD R4, R4, R1
i11:  ADD R1, R1, #1
i12:  B LOOP
     DONE:
```

A. Consider the ARM five-stage pipelined CPU with fetch, decode, execute, memory, and writeback stages. There is no forwarding and hazard detection in the hardware. Your task is to insert NOPs in the assembly code to execute the instructions correctly on the pipelined CPU. Rewrite the code above by inserting NOPs. *Your answer is only valid if you use as few NOPS as possible.* You do not need to write complete instructions. You can use instruction labels, i1, i2, etc.

B. Now consider the ARM five-stage pipelined CPU with forwarding and hazard detection in hardware. In the case of conditional branches, this CPU predicts branches are not taken. You can make the following assumptions: (1) Branches resolve and update the PC at the end of the writeback stage, (2) Memory accesses take one cycle.

Identify all pairs of instructions where forwarding helps avoid stalls altogether. Then, identify all pairs of instructions that lead to one or more stall cycles despite hazard detection.

C. How many cycles does the program take executing on the CPUs in (A) and (B)?

Q7b. Consider the following ARM assembly code. Answer the following question.

```
i1: LDR R1, [R0, #16]
i2: ADD R4, R1, #32
i3: LDR R2, [R0, #32]
i4: ADD R5, R2, #32
i5: LDR R3, [R0, #48]
i6: ADD R6, R3, #32
```

Suppose the above straight-line instruction sequence is executed on a computer with an in-order scalar pipelined CPU that stalls on a read-after-write hazard. Suppose the memory system of this computer allows four memory accesses to be performed concurrently. Reorder the above instruction sequence by hand to take advantage of this memory-level parallelism feature. Write your new code below.

Q7c. This question requires you to complete the pipeline diagram and show the contents of different structures of a scalar out-of-order processor that uses a CDC 6600-style scoreboard for resolving read-after-write hazards. CDC 6600 uses dynamic scheduling but no speculation. Use the following assumptions.

- A WAR or WAW hazard stalls instruction fetch in the RR stage.
- The CPU has an 8-entry architectural register file. The contents of the register file are shown below.
- All scoreboard entries are 1 (register is read to read) before the execution of the first instruction. A busy register, i.e., value is being computed, and dependent instruction must wait, is indicated by {v = 0} in the scoreboard. The scoreboard is shown below.
- The issue queue is empty. All slots are available (v = 0). An unavailable slot is indicated by the v bit set to 1. It stores the destination tag (dst tag), source tags or values (rs1/rs2 tag/value), and bits to indicate if source operands are ready (rs1/rs2 rdy). The issue queue is shown below.
- The pipeline is fully flushed before the execution of the first instruction.
- Any memory access results in a cache miss.
- Memory accesses are completed in three steps: (1) address generation, (2) data cache access, and (3) main memory access. The request is sent to memory during the next cycle if data is not present in the cache.
- It takes four cycles for the memory to respond. Therefore, an access that does not find the requested word in the data cache spends 1 cycle computing the address, 1 cycle looking up in the data cache, and four cycles waiting for memory (6 cycles total in the EX-stage).
- The value of the data word stored at memory location 32 is 0.

Nest, consider the four instructions below to answer the questions that follow.

```
i1: LDR R1, [R5, #0]
i2: ADD R6, R1, #4
i3: ADD R0, R2, R3
i4: ADD R7, R0, R4
```

The initial contents of the scoreboard, the register file, and the issue queue are shown below.

	v
R0	1
R1	1
R2	1
R3	1
R4	1
R5	1
R6	1
R7	1

	value
R0	#16
R1	#16
R2	#16
R3	#12
R4	#24
R5	#32
R6	#12
R7	#16

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0					
0					
0					

A. Fill in the cycle-by-cycle pipeline diagram using labels for pipeline stages below.

Fetch = FE, Decode = DE, Register Read = RR, Dispatch = DI, Issue = IS, Execute = EX, Address Generation = E@, Cache Miss/Memory access = EX$, Writeback = WB

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

B. Provide the contents of the issue queue at the end of cycle # 9. You can pick any free slot in the issue queue for any instruction. Use the following format to answer this question using text.

Cycle # X: { v=1, dst tag = `R0`, rs1 rdy = 0, rs1 tag = `R0`, rs2 rdy = 0, rs2 val = 0 }

C. Provide the contents of the scoreboard at the end of cycle # 6. You can show the scoreboard contents by writing a string of 8 binary digits, e.g., 00001111.

D. Provide the contents of the register file at the end of cycle # 9. Use the format below.

{`R0` = 10, `R1` = 10, `R2` = 13, and so on for the remaining registers}

END OF EXAMINATION